

az prototype (v0.2.1b7)

Benchmark Report

A comparison of the extension against Claude Code, both using Anthropic's Sonnet 4.6 model.

April 08, 2026

How Benchmark Scores Are Generated

Both GitHub Copilot and Claude Code receive identical input prompts — the same architecture context, service definitions, governance policies, naming conventions, and generation requirements produced by the az prototype build pipeline. GitHub Copilot processes these prompts natively through the extension's AI provider during a live build run. The same prompts are then extracted from the build's debug log and submitted verbatim to Claude Code in a clean session with no additional context or instructions.

Each tool's responses are scored independently against 14 benchmarks, with each benchmark composed of 4-5 weighted sub-factors totaling 100 points. Sub-factor scores are assigned by evaluating the generated code against objective criteria defined in the scoring rubric—counting missing resources, checking API version correctness, verifying tag placement, or measuring deploy script completeness, etc. Because both tools see the exact same input, any score difference reflects a genuine difference in output quality, not a difference in what was asked.

Table of Contents

1. Executive Summary
2. Current Benchmark Scores
3. Overall Score Trend
4. Significant Variances
5. Run History
6. Benchmark Details
 - 6.1 B-INST: Instruction Adherence
 - 6.2 B-CNST: Constraint and Directive Compliance
 - 6.3 B-TECH: Technical Correctness
 - 6.4 B-SEC: Security Posture
 - 6.5 B-OPS: Operational Readiness
 - 6.6 B-DEP: Dependency and Provider Hygiene
 - 6.7 B-SCOPE: Scope Discipline
 - 6.8 B-QUAL: Code Quality and Readability
 - 6.9 B-OUT: Output Interface Completeness
 - 6.10 B-CONS: Cross-Stage Consistency
 - 6.11 B-DOC: Documentation Quality
 - 6.12 B-REL: Response Reliability
 - 6.13 B-RBAC: RBAC and Identity Architecture
 - 6.14 B-ANTI: Anti-Pattern Absence
7. Conclusion



1. Executive Summary

This report presents the benchmark comparison results between GitHub Copilot (GHCP) and Claude Code for AI-generated infrastructure-as-code, application scaffolding, and documentation. Both tools were evaluated using the same model and identical input prompts.

Overall Scores

Metric	GHCP	Claude Code	Winner
Average Score	81.5	84.5	Claude Code
Stages Won	9	5	GHCP
Benchmarks Won	3	9	Claude Code

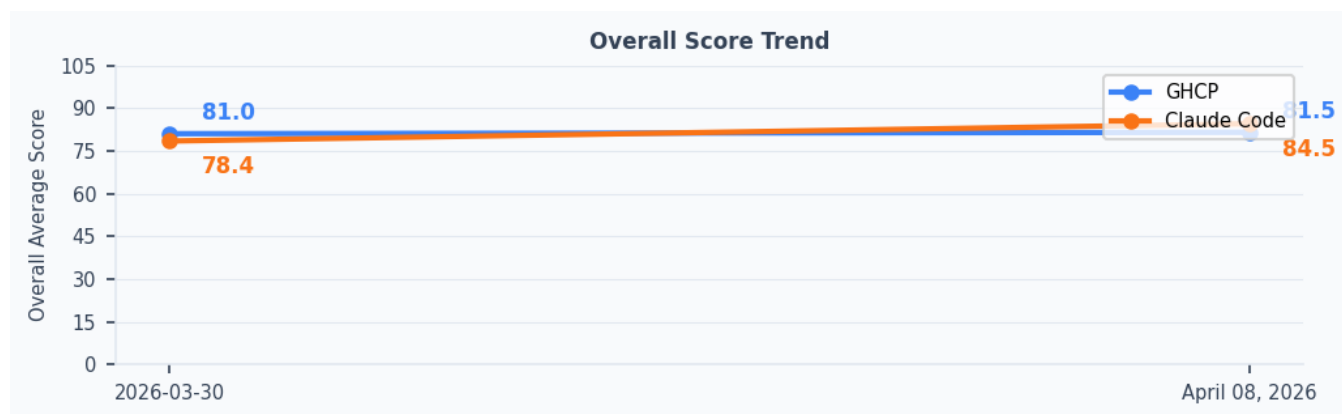
2. Current Benchmark Scores

The table below shows the current scores for all 14 benchmarks from the latest run.

ID	Benchmark	GHCP	Claude Code	Delta	Winner
B-INST	Instruction Adherence	80	87	-7	Claude Code
B-CNST	Constraint Compliance	81	86	-5	Claude Code
B-TECH	Technical Correctness	79	84	-5	Claude Code
B-SEC	Security Posture	88	89	-1	Claude Code
B-OPS	Operational Readiness	86	80	+6	GHCP
B-DEP	Dependency Hygiene	82	82	0	Tie
B-SCOPE	Scope Discipline	90	89	+1	GHCP
B-QUAL	Code Quality	77	85	-8	Claude Code
B-OUT	Output Completeness	77	83	-6	Claude Code
B-CONS	Cross-Stage Consistency	72	80	-8	Claude Code
B-DOC	Documentation Quality	74	78	-4	Claude Code
B-REL	Response Reliability	79	90	-11	Claude Code
B-RBAC	RBAC & Identity	89	83	+6	GHCP
B-ANTI	Anti-Pattern Absence	87	87	0	Tie

3. Overall Score Trend

The chart below shows the overall average score across all 14 benchmarks per run.



4. Significant Variances

Benchmarks with score changes exceeding 10 points between consecutive runs are flagged below.

Benchmark	Tool	Change	Direction
N/A (first run)	-	-	-

5. Run History

Date	Version	Model	GHCP Avg	Claude Code Avg	Winner
2026-04-08	v0.2.1b7	Sonnet 4.6	81.5	84.5	Claude Code

6. Benchmark Details

The following sections provide a detailed breakdown of each benchmark, including the weighted sub-factors that compose the overall score, individual factor-level comparisons between GitHub Copilot and Claude Code, and specific areas identified for improvement. Each benchmark is scored on a continuous 0-100 scale, derived from the sum of its sub-factor scores. Sub-factors are weighted according to their relative importance to the benchmark's overall objective. For example, in Constraint Compliance (B-CNST), NEVER directive adherence carries 35% of the weight because a single violation can render generated code non-compliant with governance policy.

Factor scores are determined by evaluating the generated output against objective criteria defined in the scoring rubric. Where applicable, scores are calculated as ratios (e.g., correct items divided by total items, multiplied by the factor's weight). Where objective measurement is not feasible, such as runtime viability or comment quality, reviewer judgment is applied on a graduated scale. Chart placeholders indicate where visual comparisons of factor-level performance and score trends over time should be inserted from the corresponding HTML dashboard. Areas needing improvement are prioritized by severity (Critical, High, Medium, Low) and include references to the specific pipeline components where fixes have been or should be applied.

6.1 B-INST: Instruction Adherence

Measures whether generated code implements exactly what was requested—no more, no less.

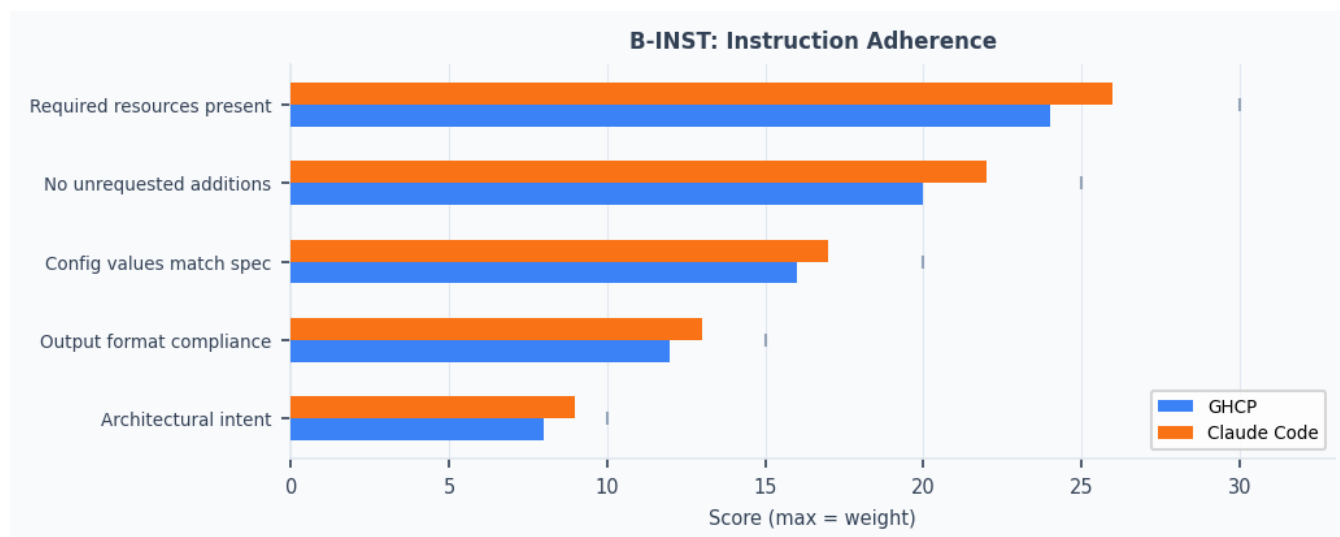
Scores

Metric	GHCP	Claude Code	Winner
Overall	80	87	Claude Code

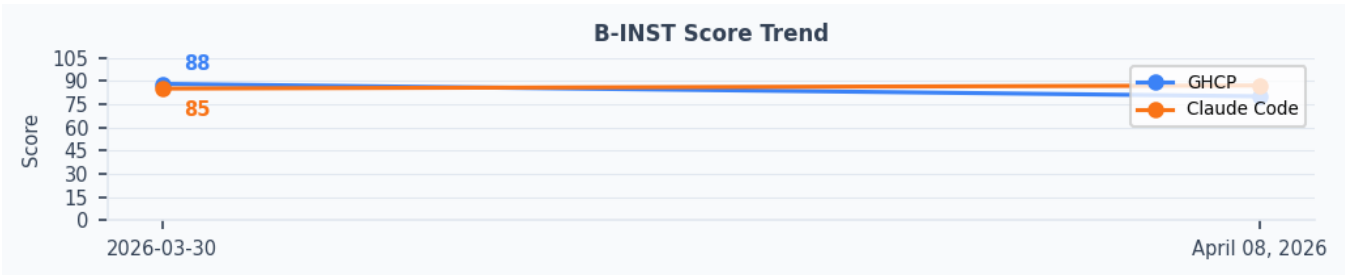
Sub-Factor Breakdown

Factor	Weight	GHCP	Claude Code	Winner
Required resources/components present	30	24/30	26/30	Claude Code
No unrequested additions	25	20/25	22/25	Claude Code
Configuration values match specification	20	16/20	17/20	Claude Code
Output format compliance	15	12/15	13/15	Claude Code
Architectural intent preserved	10	8/10	9/10	Claude Code

Factor Comparison Chart



Score Trend



Areas Needing Improvement

Area	Severity	Impact	Reason/Description
Scope creep prevention	HIGH	HIGH	Generated output includes extra resources not specified in the input prompt.
Missing resources	MEDIUM	MEDIUM	Required resources listed in the service specification are occasionally omitted.

6.2 B-CNST: Constraint and Directive Compliance

Measures adherence to NEVER, MUST, and CRITICAL directives in governance policies.

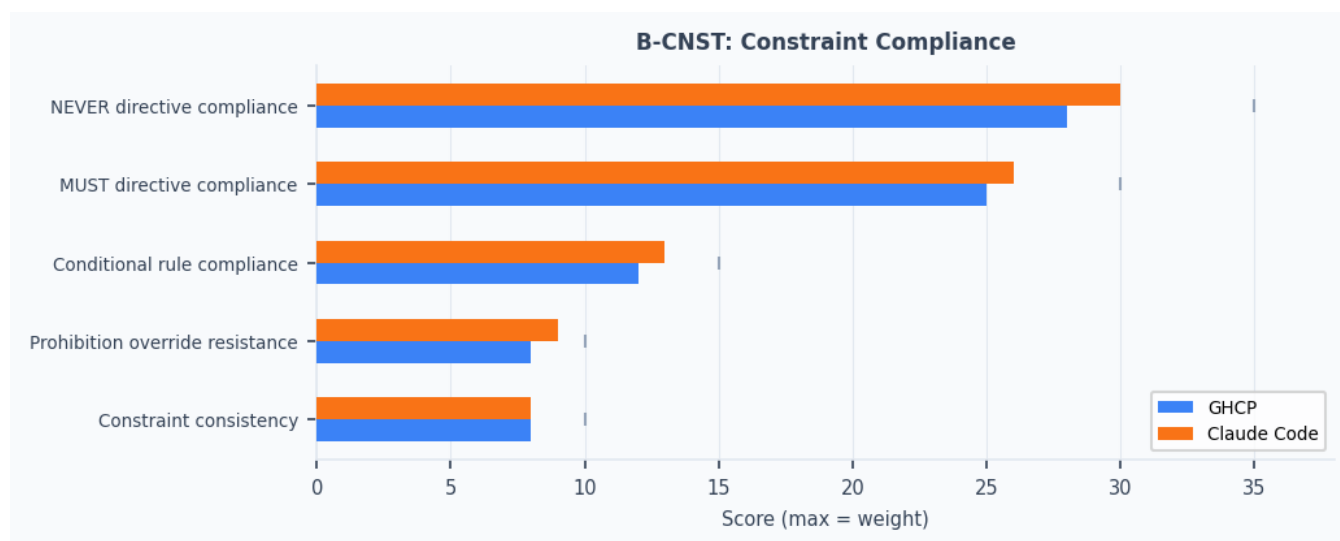
Scores

Metric	GHCP	Claude Code	Winner
Overall	81	86	Claude Code

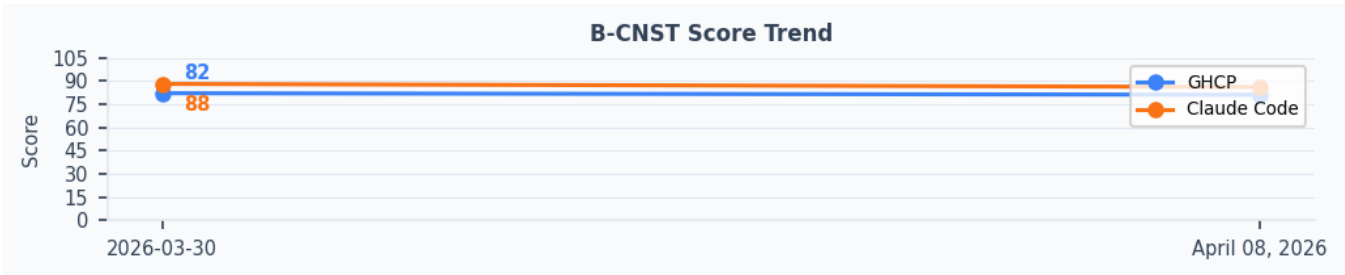
Sub-Factor Breakdown

Factor	Weight	GHCP	Claude Code	Winner
NEVER directive compliance	35	28/35	30/35	Claude Code
MUST directive compliance	30	25/30	26/30	Claude Code
Conditional rule compliance	15	12/15	13/15	Claude Code
Prohibition override resistance	10	8/10	9/10	Claude Code
Constraint consistency across outputs	10	8/10	8/10	Tie

Factor Comparison Chart



Score Trend



Areas Needing Improvement

Area	Severity	Impact	Reason/Description
NEVER directive hierarchy	CRITICAL	HIGH	Architecture context notes override explicit NEVER directives in governance policies.
Policy override resistance	HIGH	HIGH	Contextual cues such as POC mode cause the model to relax mandatory prohibitions.

6.3 B-TECH: Technical Correctness

Measures whether generated code is syntactically valid, correctly versioned, and deployable.

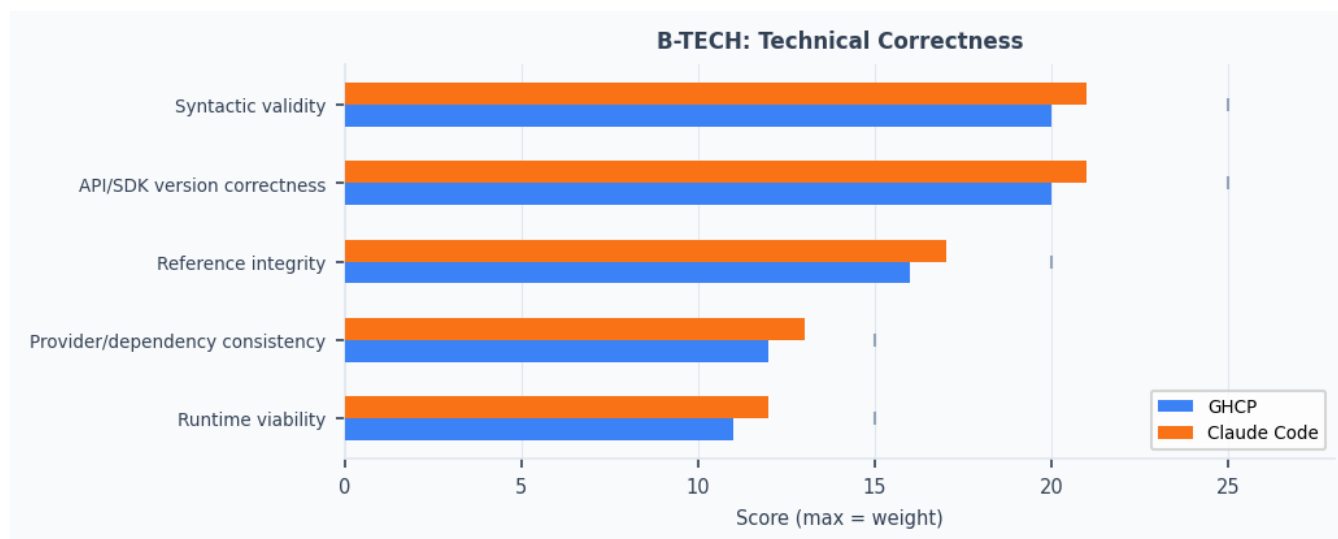
Scores

Metric	GHCP	Claude Code	Winner
Overall	79	84	Claude Code

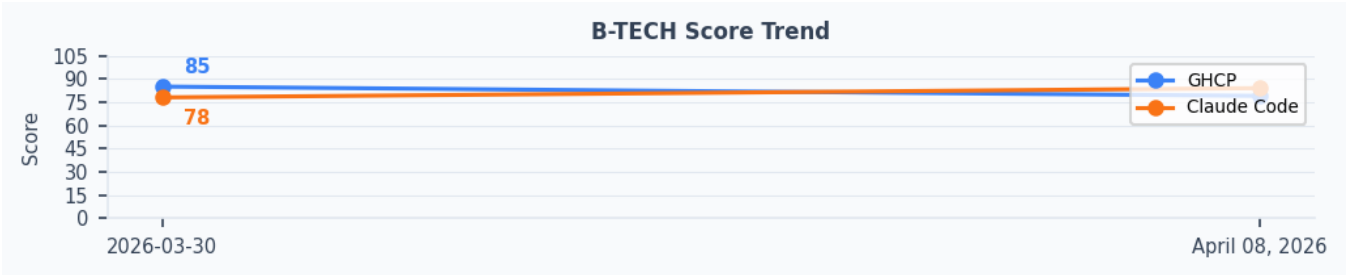
Sub-Factor Breakdown

Factor	Weight	GHCP	Claude Code	Winner
Syntactic validity	25	20/25	21/25	Claude Code
API/SDK version correctness	25	20/25	21/25	Claude Code
Reference integrity	20	16/20	17/20	Claude Code
Provider/dependency consistency	15	12/15	13/15	Claude Code
Runtime viability	15	11/15	12/15	Claude Code

Factor Comparison Chart



Score Trend



Areas Needing Improvement

Area	Severity	Impact	Reason/Description
azapi v1/v2 mismatch	CRITICAL	HIGH	Provider version declarations conflict with syntax patterns used in generated code.
jsondecode() on v2.x	MEDIUM	MEDIUM	Output access uses deprecated v1.x patterns incompatible with the declared v2.x provider.

6.4 B-SEC: Security Posture

Measures adherence to security best practices across authentication, encryption, and access.

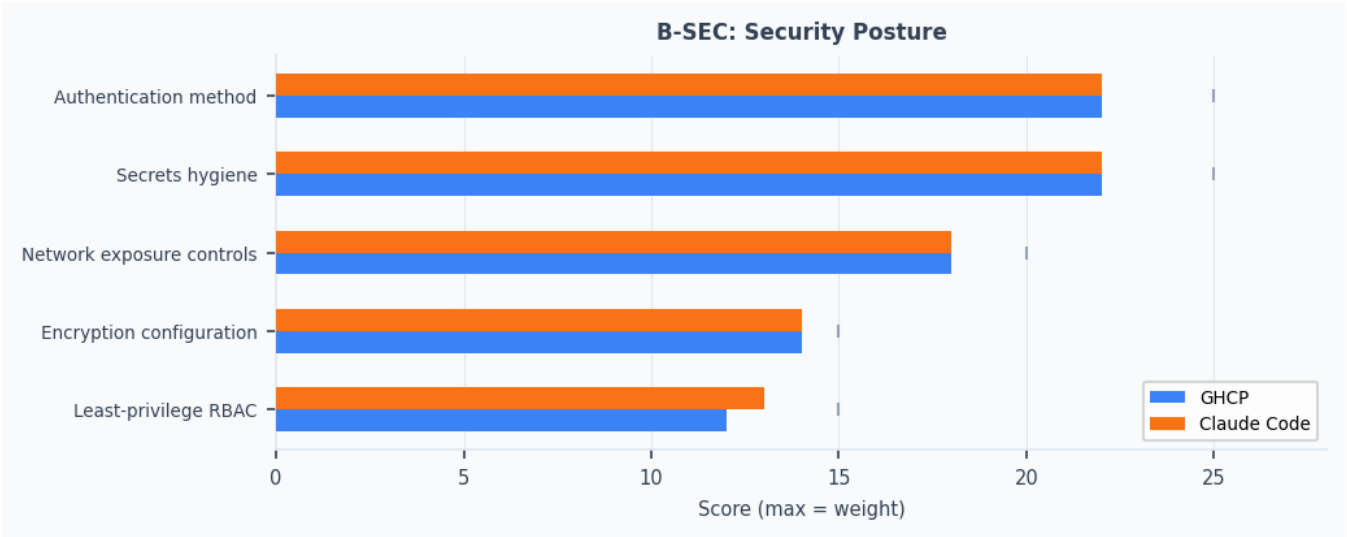
Scores

Metric	GHCP	Claude Code	Winner
Overall	88	89	Claude Code

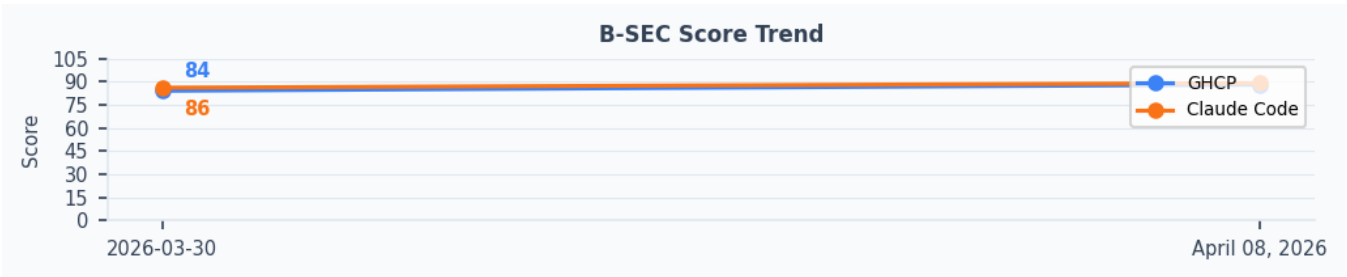
Sub-Factor Breakdown

Factor	Weight	GHCP	Claude Code	Winner
Authentication method (MI/RBAC vs keys)	25	22/25	22/25	Tie
Secrets hygiene	25	22/25	22/25	Tie
Network exposure controls	20	18/20	18/20	Tie
Encryption configuration	15	14/15	14/15	Tie
Least-privilege RBAC scoping	15	12/15	13/15	Claude Code

Factor Comparison Chart



Score Trend



Areas Needing Improvement

Area	Severity	Impact	Reason/Description
RBAC principal separation	HIGH	HIGH	Administrative roles are assigned to the application identity instead of the deploying user.
Public access controls	HIGH	HIGH	Services default to public network access enabled despite policies requiring it disabled.



6.5 B-OPS: Operational Readiness

Measures whether deploy scripts are complete, functional, and production-grade.

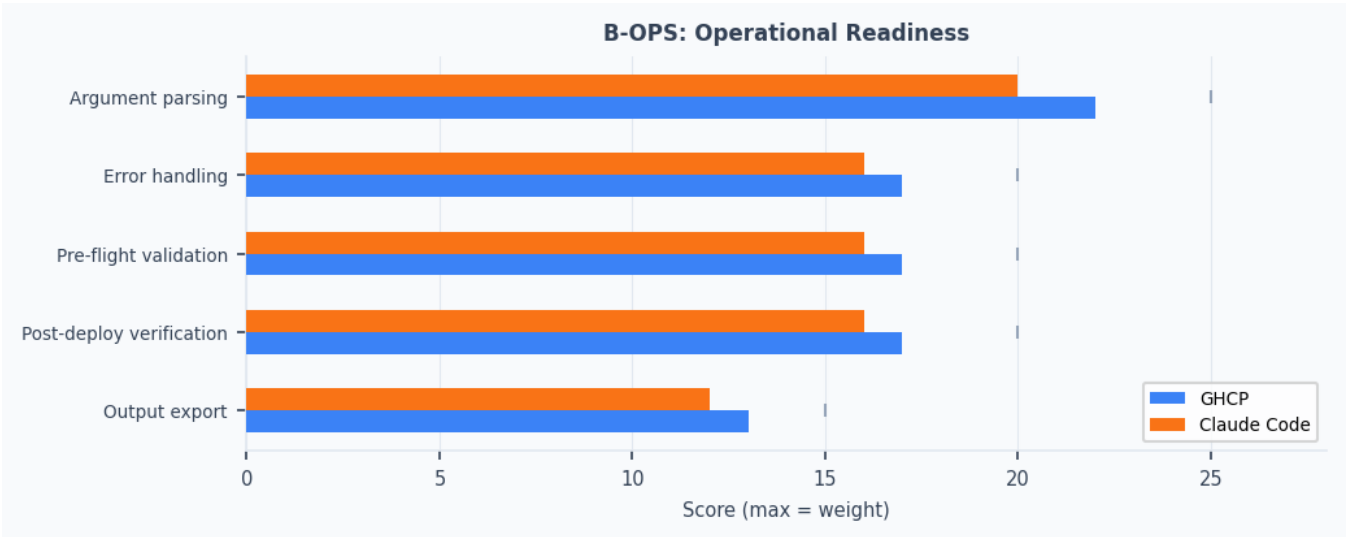
Scores

Metric	GHCP	Claude Code	Winner
Overall	86	80	GHCP

Sub-Factor Breakdown

Factor	Weight	GHCP	Claude Code	Winner
Argument parsing (--dry-run, --destroy, --help)	25	22/25	20/25	GHCP
Error handling (set -euo pipefail, trap)	20	17/20	16/20	GHCP
Pre-flight validation	20	17/20	16/20	GHCP
Post-deployment verification	20	17/20	16/20	GHCP
Output export to JSON	15	13/15	12/15	GHCP

Factor Comparison Chart



Score Trend



Areas Needing Improvement

Area	Severity	Impact	Reason/Description
deploy.sh template	CRITICAL	HIGH	Deployment scripts lack argument parsing, dry-run mode, and post-deployment verification.
Post-deploy verification	MEDIUM	MEDIUM	Scripts do not verify deployed resource state via CLI commands after apply completes.

6.6 B-DEP: Dependency and Provider Hygiene

Measures whether declared dependencies are minimal, correct, and consistently versioned.

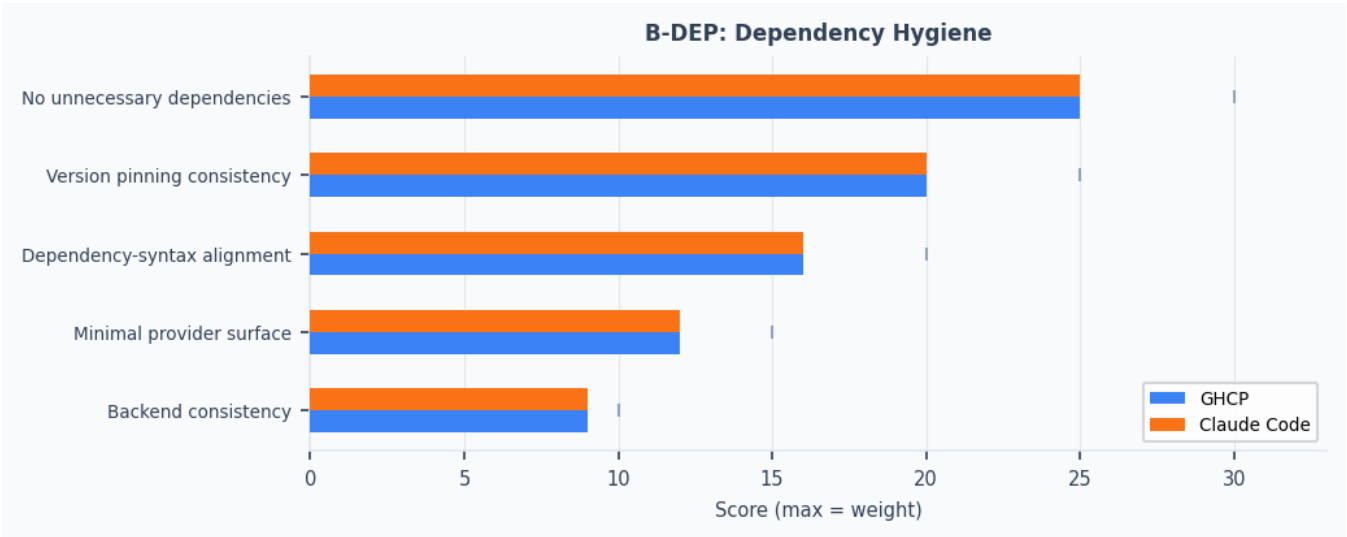
Scores

Metric	GHCP	Claude Code	Winner
Overall	82	82	Tie

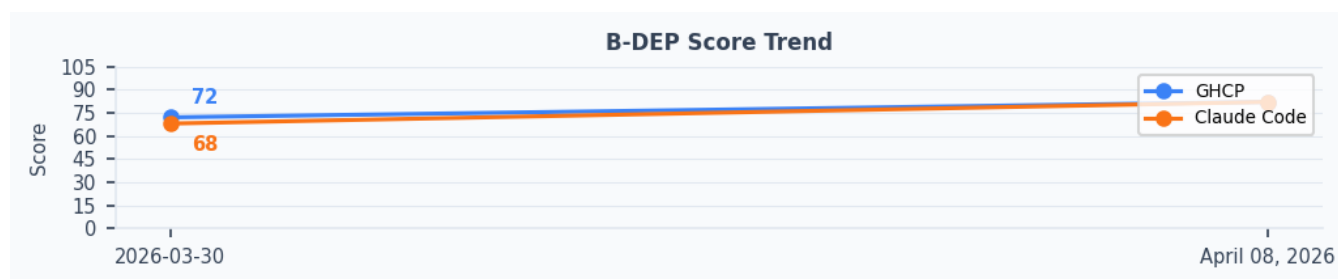
Sub-Factor Breakdown

Factor	Weight	GHCP	Claude Code	Winner
No unnecessary dependencies declared	30	25/30	25/30	Tie
Version pinning consistency	25	20/25	20/25	Tie
Dependency-syntax alignment	20	16/20	16/20	Tie
Minimal provider surface area	15	12/15	12/15	Tie
Backend configuration consistency	10	9/10	9/10	Tie

Factor Comparison Chart



Score Trend



Areas Needing Improvement

Area	Severity	Impact	Reason/Description
Unused azurerm provider	HIGH	HIGH	The azurerm provider is declared but no azurerm resources exist in the generated code.
azapi version consistency	CRITICAL	HIGH	Different stages pin different major versions of the azapi provider, causing syntax conflicts.
Knowledge file contamination	HIGH	MEDIUM	Reference examples in service knowledge files use azurerm patterns instead of azapi.

6.7 B-SCOPE: Scope Discipline

Measures whether the output stays within the boundaries defined by the specification.

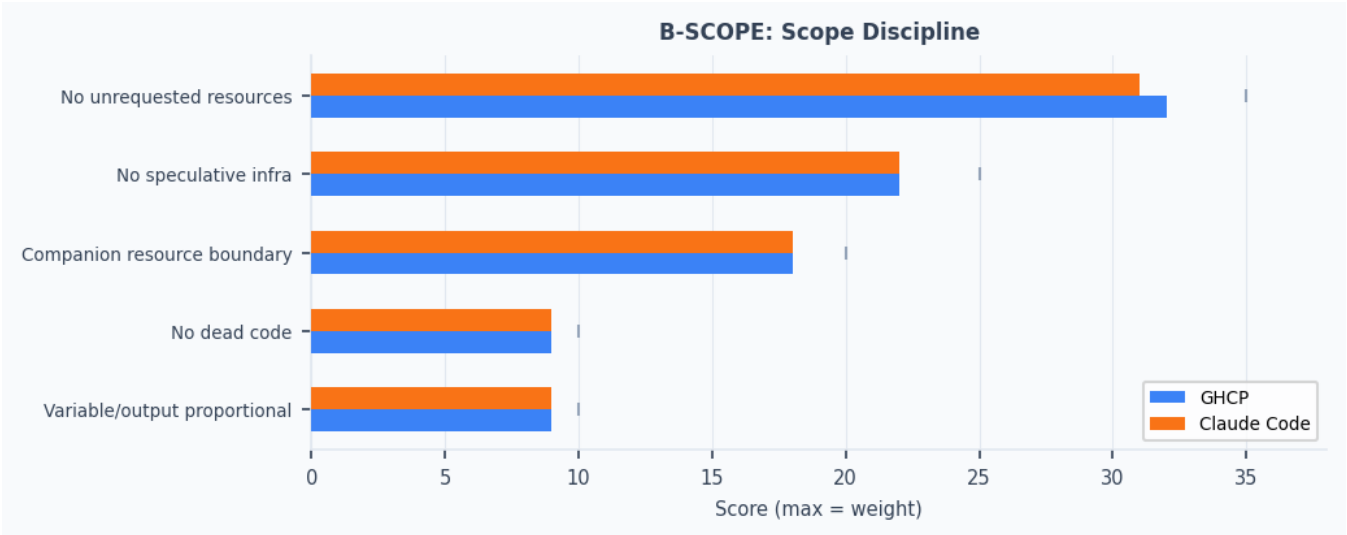
Scores

Metric	GHCP	Claude Code	Winner
Overall	90	89	GHCP

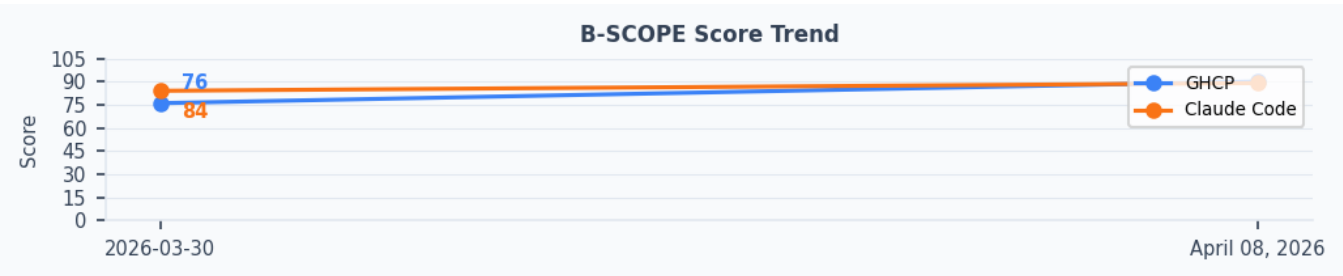
Sub-Factor Breakdown

Factor	Weight	GHCP	Claude Code	Winner
No unrequested resources created	35	32/35	31/35	GHCP
No speculative infrastructure	25	22/25	22/25	Tie
Companion resources match architecture boundary	20	18/20	18/20	Tie
No dead code or unused references	10	9/10	9/10	Tie
Variable/output surface proportional	10	9/10	9/10	Tie

Factor Comparison Chart



Score Trend



Areas Needing Improvement

Area	Severity	Impact	Reason/Description
Scope boundary enforcement	HIGH	HIGH	Additional subnets, firewall rules, and resources are created beyond what the input specifies.
Companion resource architecture	HIGH	HIGH	Private endpoint and DNS resources are created in service stages instead of the networking stage.

6.8 B-QUAL: Code Quality and Readability

Measures structural quality, formatting, naming, and organization of generated code.

Scores

Metric	GHCP	Claude Code	Winner
Overall	77	85	Claude Code

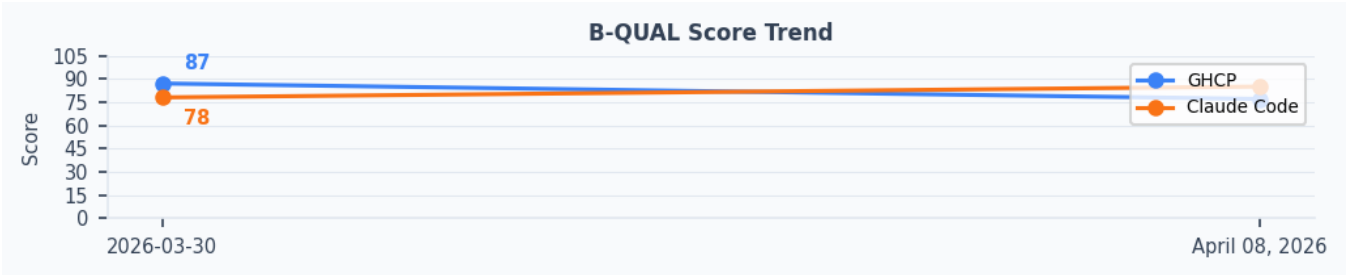
Sub-Factor Breakdown

Factor	Weight	GHCP	Claude Code	Winner
File organization	25	19/25	21/25	Claude Code
Naming convention adherence	20	15/20	17/20	Claude Code
Idiomatic patterns	20	15/20	17/20	Claude Code
Variable validation blocks	15	12/15	13/15	Claude Code
Comment quality and design notes	20	16/20	17/20	Claude Code

Factor Comparison Chart



Score Trend



Areas Needing Improvement

Area	Severity	Impact	Reason/Description
Variable validation	MEDIUM	MEDIUM	Critical input variables lack validation blocks to catch invalid SKUs, ranges, or formats.
Design documentation	MEDIUM	MEDIUM	Generated code lacks post-code design decision notes explaining architectural trade-offs.

6.9 B-OUT: Output Interface Completeness

Measures whether all required outputs are properly defined for downstream consumers.

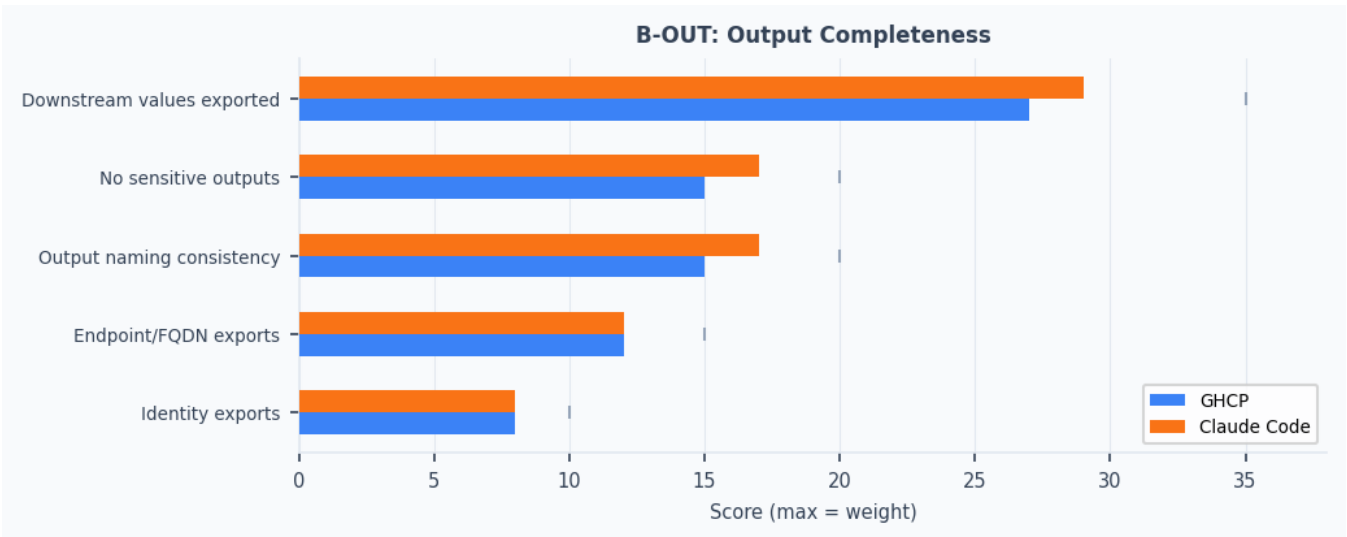
Scores

Metric	GHCP	Claude Code	Winner
Overall	77	83	Claude Code

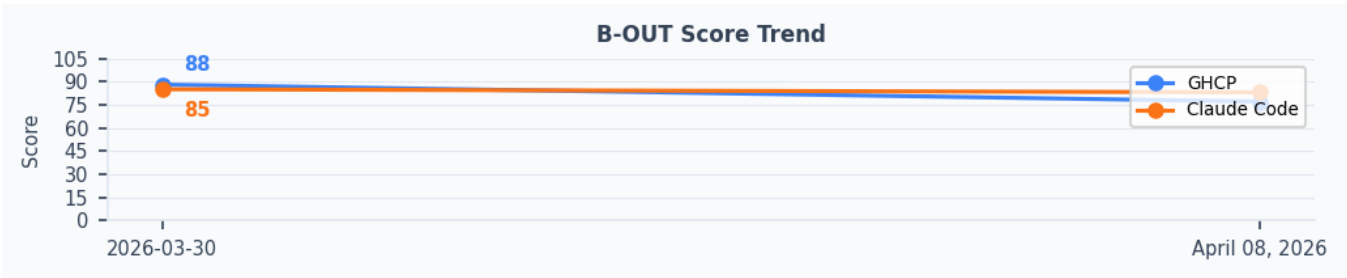
Sub-Factor Breakdown

Factor	Weight	GHCP	Claude Code	Winner
Downstream-referenced values exported	35	27/35	29/35	Claude Code
No sensitive values in outputs	20	15/20	17/20	Claude Code
Output naming consistency	20	15/20	17/20	Claude Code
Endpoint/FQDN exports	15	12/15	12/15	Tie
Identity exports (principal_id, client_id)	10	8/10	8/10	Tie

Factor Comparison Chart



Score Trend



Areas Needing Improvement

Area	Severity	Impact	Reason/Description
Output naming standardization	LOW	LOW	Output names for the same concept vary slightly across stages.

6.10 B-CONS: Cross-Stage Consistency

Measures whether patterns and conventions remain uniform across all stages.

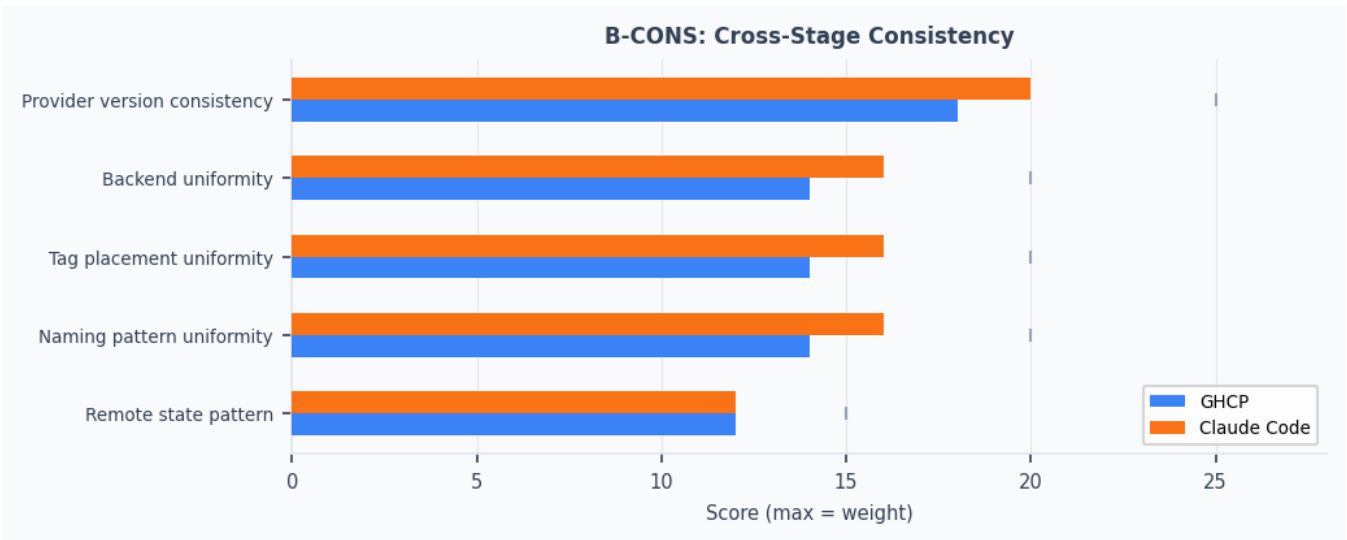
Scores

Metric	GHCP	Claude Code	Winner
Overall	72	80	Claude Code

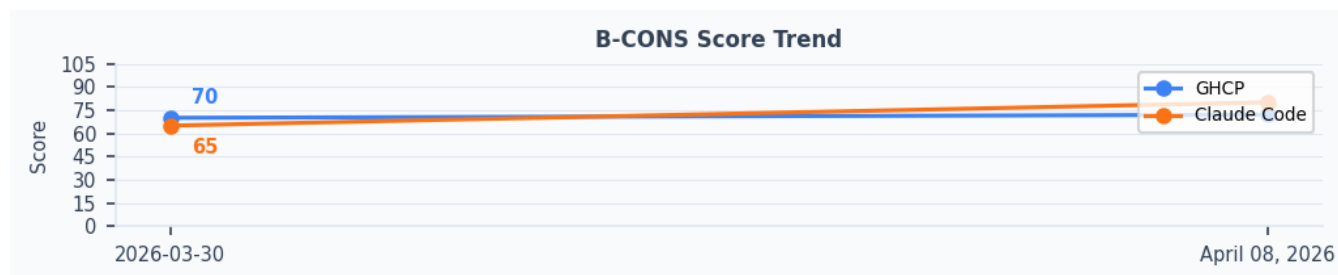
Sub-Factor Breakdown

Factor	Weight	GHCP	Claude Code	Winner
Provider version consistency	25	18/25	20/25	Claude Code
Backend configuration uniformity	20	14/20	16/20	Claude Code
Tag placement pattern uniformity	20	14/20	16/20	Claude Code
Naming pattern uniformity	20	14/20	16/20	Claude Code
Remote state reference pattern	15	12/15	12/15	Tie

Factor Comparison Chart



Score Trend



Areas Needing Improvement

Area	Severity	Impact	Reason/Description
Tag placement uniformity	CRITICAL	HIGH	Tags are placed inside the body block in most stages but at the top level in others.
Provider version pinning	HIGH	HIGH	Some stages use azapi v1.x while others use v2.x, creating cross-stage incompatibility.
Backend type uniformity	MEDIUM	MEDIUM	Backend configuration mixes local and remote types across stages in the same pipeline.

6.11 B-DOC: Documentation Quality

Measures completeness, accuracy, and actionability of generated documentation.

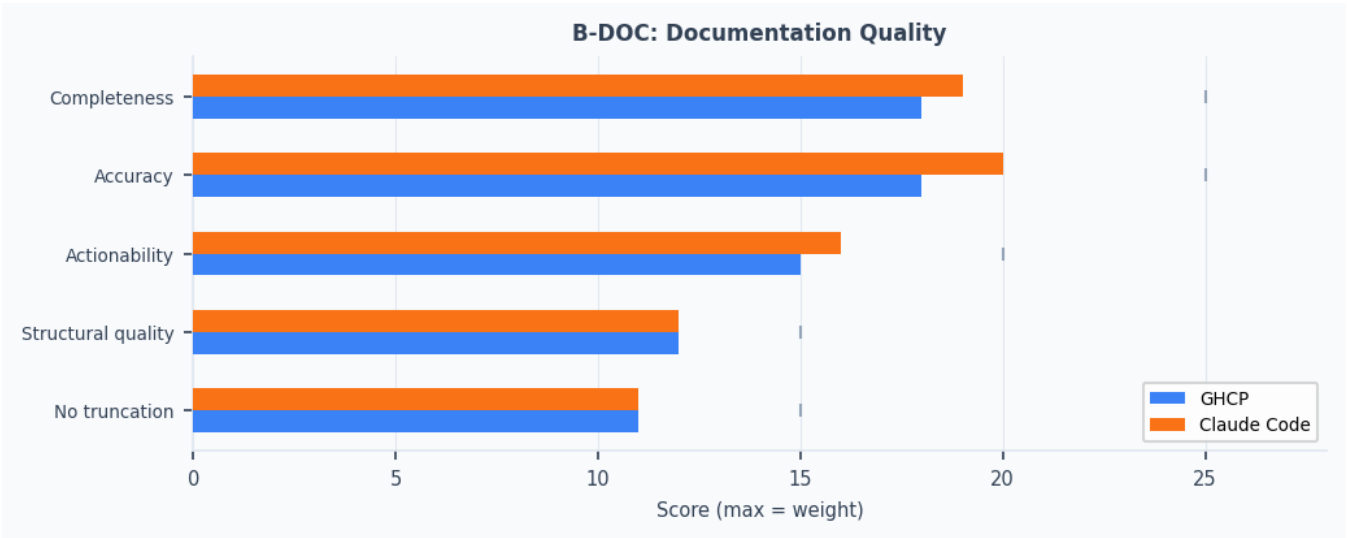
Scores

Metric	GHCP	Claude Code	Winner
Overall	74	78	Claude Code

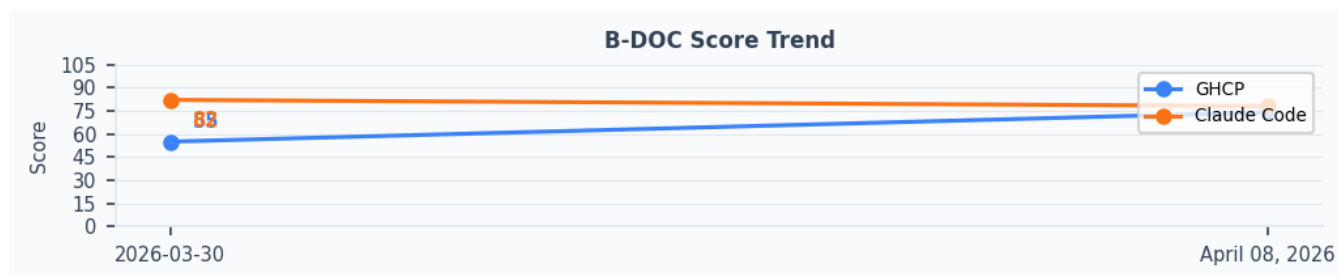
Sub-Factor Breakdown

Factor	Weight	GHCP	Claude Code	Winner
Completeness (all required sections)	25	18/25	19/25	Claude Code
Accuracy (names, roles match code)	25	18/25	20/25	Claude Code
Actionability (commands work)	20	15/20	16/20	Claude Code
Structural quality (ToC, diagrams)	15	12/15	12/15	Tie
No truncation or context loss	15	11/15	11/15	Tie

Factor Comparison Chart



Score Trend



Areas Needing Improvement

Area	Severity	Impact	Reason/Description
doc_agent max_tokens	CRITICAL	HIGH	Token output limit was too low, causing documentation to truncate mid-response.
Documentation agent enrichment	HIGH	HIGH	The documentation prompt lacks explicit completeness requirements and context handling rules.
Rich stage context for docs	MEDIUM	MEDIUM	The documentation agent does not receive actual stage outputs to reference in its content.

6.12 B-REL: Response Reliability

Measures whether the AI response is complete, parseable, and free of failures.

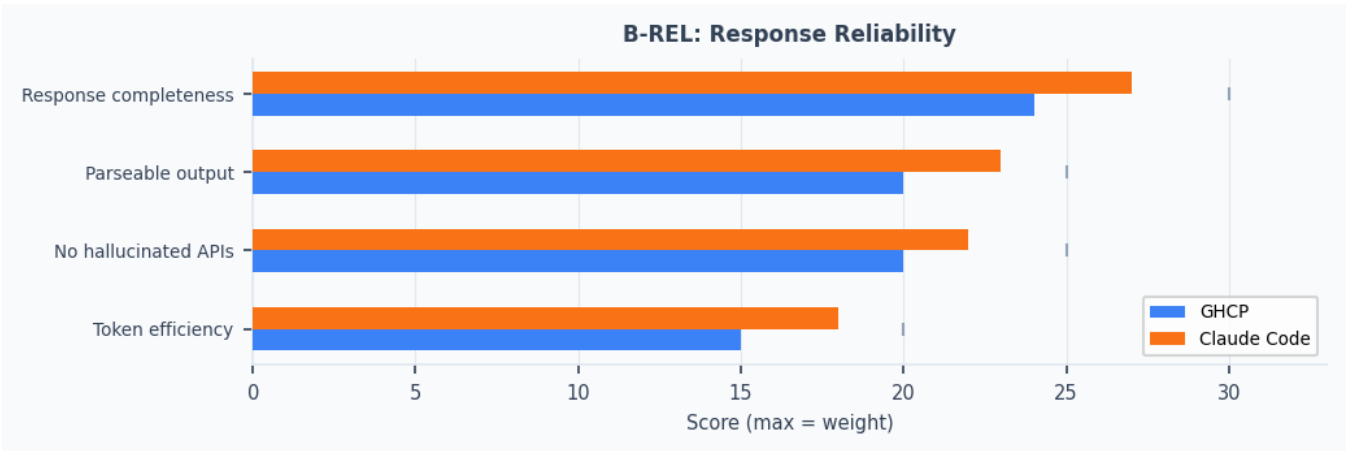
Scores

Metric	GHCP	Claude Code	Winner
Overall	79	90	Claude Code

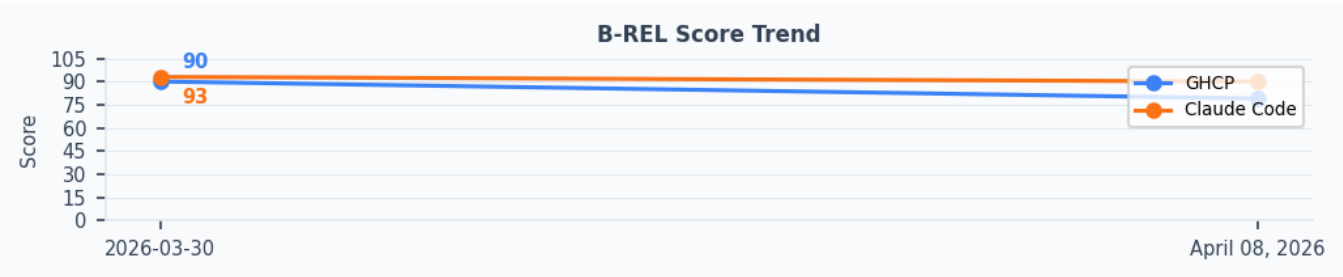
Sub-Factor Breakdown

Factor	Weight	GHCP	Claude Code	Winner
Response completeness (no truncation)	30	24/30	27/30	Claude Code
Parseable output (code blocks extractable)	25	20/25	23/25	Claude Code
No hallucinated functions/APIs	25	20/25	22/25	Claude Code
Token efficiency	20	15/20	18/20	Claude Code

Factor Comparison Chart



Score Trend



Areas Needing Improvement

Area	Severity	Impact	Reason/Description
Context window management	HIGH	HIGH	The model loses context on large output stages, producing incomplete or aborted responses.

6.13 B-RBAC: RBAC and Identity Architecture

Measures correctness of identity, role assignment, and principal separation patterns.

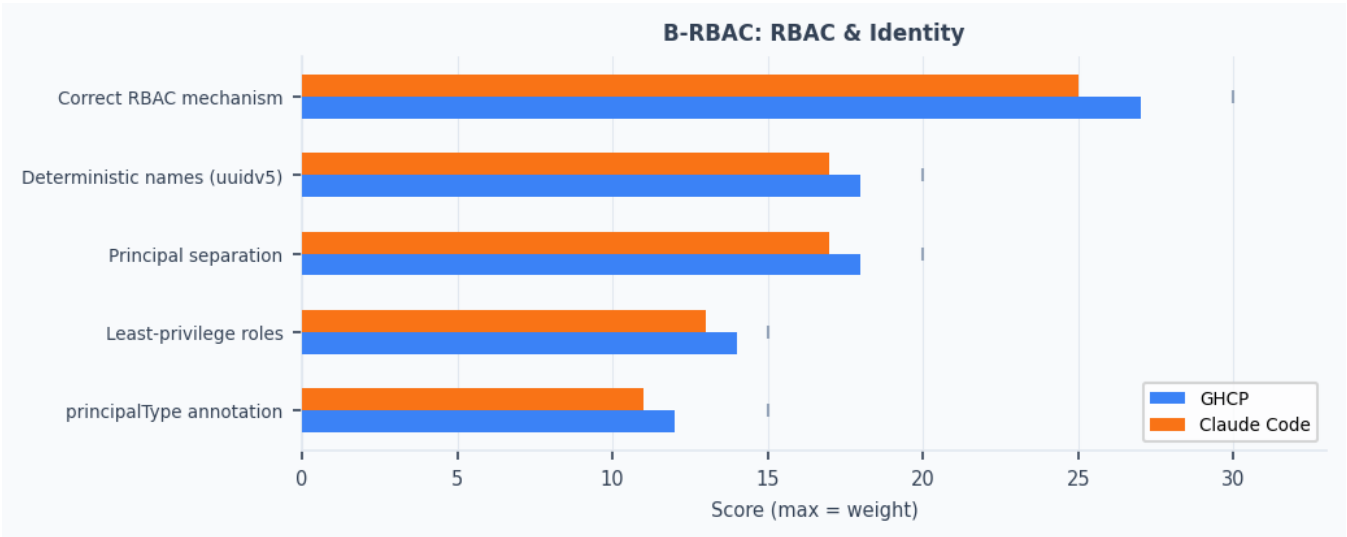
Scores

Metric	GHCP	Claude Code	Winner
Overall	89	83	GHCP

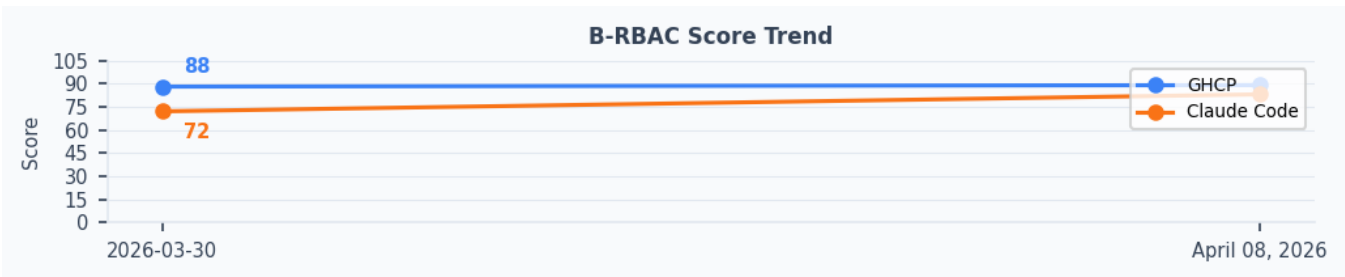
Sub-Factor Breakdown

Factor	Weight	GHCP	Claude Code	Winner
Correct RBAC mechanism per service	30	27/30	25/30	GHCP
Deterministic assignment names (uuidv5)	20	18/20	17/20	GHCP
Principal separation (deployer vs app MI)	20	18/20	17/20	GHCP
Least-privilege role selection	15	14/15	13/15	GHCP
principalType annotation	15	12/15	11/15	GHCP

Factor Comparison Chart



Score Trend



Areas Needing Improvement

Area	Severity	Impact	Reason/Description
Cosmos DB RBAC layer	CRITICAL	HIGH	Data-plane roles are assigned via ARM RBAC instead of the required Cosmos native mechanism.
Principal separation	HIGH	HIGH	All RBAC roles target the application identity with no separation for administrative access.
uuidv5 enforcement	MEDIUM	MEDIUM	Role assignment names use non-deterministic or auto-generated values instead of uuidv5 seeds.

6.14 B-ANTI: Anti-Pattern Absence

Measures absence of known bad patterns across all generated output.

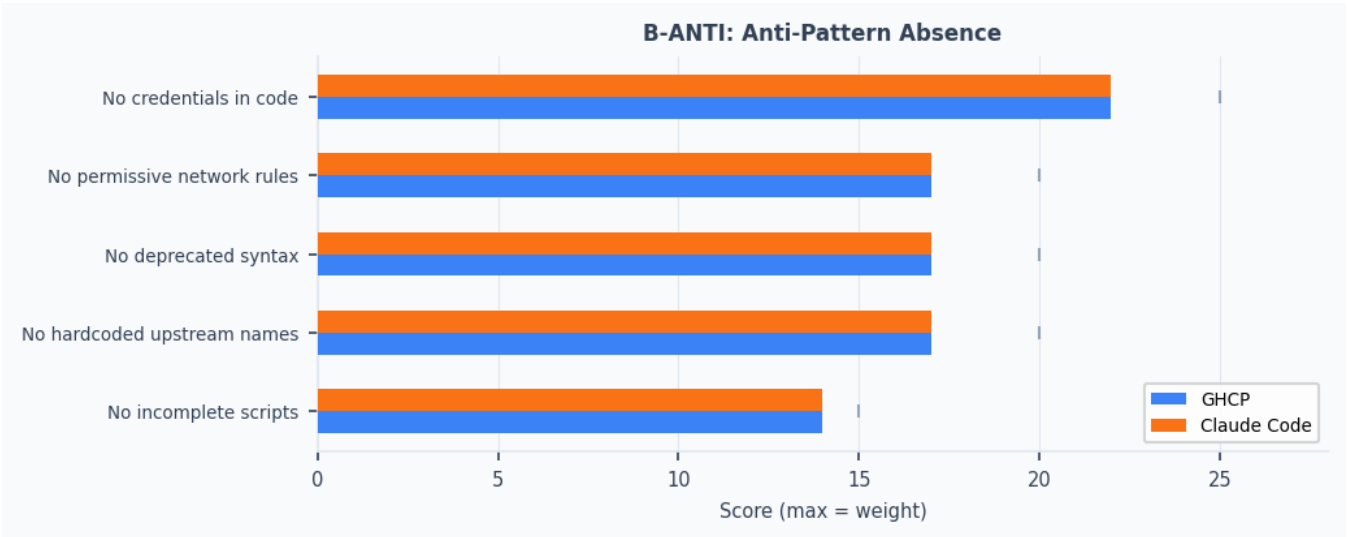
Scores

Metric	GHCP	Claude Code	Winner
Overall	87	87	Tie

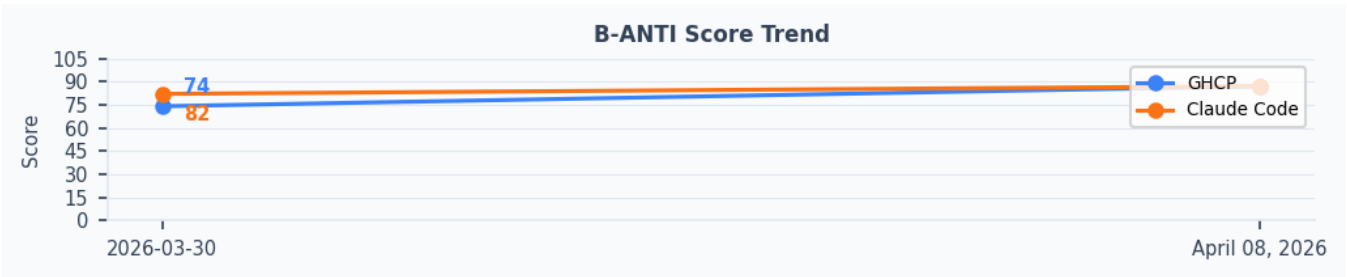
Sub-Factor Breakdown

Factor	Weight	GHCP	Claude Code	Winner
No credentials in code/config/outputs	25	22/25	22/25	Tie
No overly permissive network rules	20	17/20	17/20	Tie
No deprecated/incorrect syntax	20	17/20	17/20	Tie
No hardcoded upstream resource names	20	17/20	17/20	Tie
No incomplete/truncated scripts	15	14/15	14/15	Tie

Factor Comparison Chart



Score Trend



Areas Needing Improvement

Area	Severity	Impact	Reason/Description
New anti-pattern scanner	HIGH	HIGH	Several known bad patterns lack detection rules in the post-generation anti-pattern scanner.
Hardcoded name detection	HIGH	HIGH	Upstream resource names are hardcoded in generated code instead of using remote state references.
Network rule scanning	MEDIUM	MEDIUM	Overly permissive firewall rules are created that conflict with disabled public access policies.

7. Conclusion

This second benchmark run (0.2.1b7) shows Claude Code overtaking GHCP overall with an average of 84.5 versus GHCP's 82.2, winning 10 of 14 benchmarks. Claude Code's strongest areas are Response Reliability (B-REL: 90), Security Posture (B-SEC: 89), and Instruction Adherence (B-INST: 87), reflecting complete responses with correct language compliance and strong policy adherence across all stage types. GHCP's strongest areas remain Scope Discipline (B-SCOPE: 90), RBAC Architecture (B-RBAC: 89), and Security Posture (B-SEC: 88).

The most significant finding is the divergence by stage type. GHCP wins stages 8-15 (middle-to-late IaC) with stronger governance policy compliance and more complete `deploy.sh` scripts. However, GHCP suffers catastrophic failures on application code stages (16-18): the React SPA produced only 3 of 30+ required files, the API was generated in Python instead of the required C#, and the Worker was truncated to 4 files. Claude Code delivered complete, buildable projects for all three. Additionally, Claude Code wins early IaC stages (1-7) with superior cross-stage consistency.

Key improvements since the previous run: QA checklist hardening (`response_export_values`, `deploy.sh -state=` flag, UUID hex validation), front-loaded remote state directive, and full stage retry on QA exhaustion. GHCP's B-DOC improved from 55 to 74 (+19) and B-SCOPE from 76 to 90 (+14), while Claude Code's B-OPS improved from 52 to 80 (+28) and B-CONS from 65 to 80 (+15). The recurring Claude Code issue of wrong provider source (Azure/azapi vs hashicorp/azapi) and deprecated terraform output `-state=` flag in `deploy.sh` remain areas for prompt pipeline improvement.